

Práctica 2: **SecureBox** – Cifrado híbrido, firmas y canal seguro con *Python* + cryptography

Ficha

- **Tipo:** Individual
- **Tiempo estimado:** ~20 horas (desarrollo, pruebas y documentación)
- **Plazo de entrega:** 4 semanas
- **Lenguaje:** Python 3.10+ (recomendado 3.11)
- **Librería**
obligatoria: cryptography (documentación: <https://cryptography.io/en/latest/>)

Fundamento:

En sistemas reales, la clave pública se usa para **intercambiar/proteger claves** y/o **firmar**, y el “cifrado de datos” se hace con cifrado simétrico autenticado (AEAD). Este proyecto te obliga a montar ese flujo completo, de punta a punta, con buenas prácticas, pruebas y documentación.

1) Objetivo general

Implementar **SecureBox**, una herramienta por línea de comandos (CLI) que permita:

1. **Gestión de identidades criptográficas**
 - Generación de claves **RSA** (2048 o 3072) y claves de **curva elíptica** (mínimo una opción moderna: X25519 para ECDH y/o Ed25519 para firma).
 - Serialización a **PEM** y almacenamiento **cifrado** de clave privada con contraseña.
 - Carga de claves desde disco y validaciones básicas.
2. **Cifrado híbrido de archivos (“envelope encryption”)**
 - Cifrar un archivo cualquiera usando una clave simétrica aleatoria (p. ej. **AES-GCM** o **ChaCha20-Poly1305**).

- Proteger la clave simétrica mediante **RSA-OAEP** o un mecanismo ECC (p. ej. ECDH + HKDF).
- Empaquetar todo en un **contenedor** (formato definido por ti) que incluya metadatos, versión, algoritmos y valores necesarios.

3. Firmas digitales y verificación

- Firmar el contenedor (o su “manifest”) con **RSA-PSS** o **Ed25519/ECDSA**.
- Verificar firmas y detectar manipulación.

4. Canal seguro tipo “mini-handshake”

- Diseñar un protocolo mínimo entre Cliente y Servidor (o Alice/Bob) usando **ECDHE (X25519 recomendado)** para acordar claves de sesión.
- Derivar claves con **HKDF** y cifrar mensajes con **AEAD**.
- Incluir “transcript binding”: firmar o autenticar el *handshake transcript* para evitar MITM.

5. Documentación + pruebas

- Pruebas unitarias y de integración que cubran casos correctos y fallos (clave errónea, ciphertext modificado, firma inválida, replay, etc.).
- Informe técnico con decisiones, justificación y resultados.

2) Requisitos funcionales (obligatorios)

2.1. CLI y estructura del proyecto

Debes entregar un proyecto con estructura similar a:

```
securebox/  
  pyproject.toml (o requirements.txt)  
securebox/  
  __init__.py  
  cli.py  
  keys.py  
  crypto/  
    hybrid.py  
    aead.py  
    kdf.py  
    signatures.py  
    formats.py
```

```
    handshake.py
utils/
    encoding.py
    io.py
tests/
    test_keys.py
    test_hybrid.py
    test_signatures.py
    test_handshake.py
docs/
    INFORME.pdf
    USO.md
```

La CLI debe exponer **al menos** estos comandos:

- `securebox keygen ...`
- `securebox encrypt ...`
- `securebox decrypt ...`
- `securebox sign ...`
- `securebox verify ...`
- `securebox handshake-demo ...` (simulación local)
- `securebox inspect ...` (muestra metadatos del contenedor)

2.2. Gestión de claves

Implementa estas funciones (o equivalentes) con documentación:

- `gen_rsa_private_key(public_exponent=65537, key_size=2048|3072)`
- `gen_ecdh_keypair()` (X25519 recomendado)
- `gen_sign_keypair()` (Ed25519 recomendado, o ECDSA P-256)
- `pem_serialize_public_key(pk)`
- `pem_serialize_encrypted_private_key(sk, password_bytes)`
- `pem_load_public_key(pem_bytes)`
- `pem_load_private_key(pem_bytes, password_bytes)`

Requisitos:

- La clave privada en disco debe estar **cifrada** (BestAvailableEncryption).
- Debes incluir comprobaciones de tipo de clave (no aceptar “cualquier cosa” sin validar).

2.3. Contenedor SecureBox (formato)

Define un formato de contenedor “.sbox” (tú lo decides), pero debe cumplir:

- Ser **autosuficiente**: contiene todo lo necesario para descifrar/verificar (excepto la clave privada del receptor).

- Incluir una versión (p. ej. "sbox-1").
- Incluir identificadores de algoritmos (p. ej. rsa_oaep_sha256, aes_256_gcm, x25519_hkdf_sha256, ed25519, etc.).
- Incluir recipient o key_id (puede ser huella SHA-256 de la clave pública).
- Incluir nonce/iv, ciphertext, tag (si aplica), y el material de encapsulación (p. ej. wrapped_key o ephemeral_pubkey).
- Incluir un campo aad o un manifiesto autenticado (recomendado).

Sugerencia: usa JSON + Base64 para legibilidad, pero también puedes usar CBOR.

2.4. Cifrado híbrido (dos modos obligatorios)

Tu herramienta debe soportar **dos modos** de cifrado híbrido:

Modo A — RSA Envelope (obligatorio)

- Genera una clave aleatoria K (p. ej. 32 bytes si AES-256).
- Cifra el archivo con AEAD (AESGCM o ChaCha20Poly1305).
- Cifra/"envuelve" K con **RSA-OAEP** (hash SHA-256 recomendado).
- Empaqueta wrapped_key, nonce, ciphertext y metadatos.

Modo B — ECC KEM/DEM (obligatorio)

- Usa **ECDH efímero**: genera una clave efímera del emisor (X25519) y calcula un secreto compartido con la pública del receptor.
- Usa **HKDF** para derivar K desde el secreto compartido + salt + info (incluye versión y contexto).
- Cifra el archivo con AEAD como en el Modo A.
- Empaqueta ephemeral_public_key, salt, nonce, ciphertext, metadatos.

Nota: Este modo reproduce la idea de ECDHE/ECIES/HPKE a pequeña escala (sin pedirte implementar HPKE entero).

2.5. Firmas digitales

Debes permitir firmar y verificar el contenedor .sbox.

- Firma como mínimo:
 - Opción 1: **RSA-PSS** (SHA-256)

- Opción 2: **Ed25519** (recomendado)
- Define qué se firma:
 - Recomendado: firma un manifest canonicalizado (p. ej. JSON ordenado) **que incluya** los campos críticos (algoritmos, ids, nonce, ciphertext, etc.).
- La verificación debe fallar si cualquier campo firmado cambia.

2.6. Canal seguro (handshake + mensajes)

Implementa una demo local (puede ser simulación sin sockets, o sockets locales) con este flujo mínimo:

1. **Intercambio efímero**: cada parte genera X25519 efímera y envía su pública.
2. **Derivación de claves**: ambos calculan secreto compartido y derivan claves con HKDF.
3. **Autenticación**: evita MITM vinculando el transcript:
 - O bien con firma: cada parte firma el transcript con su clave de firma (Ed25519/ECDSA/RSA-PSS).
 - O bien con un “PSK” precompartido (menos recomendado, pero aceptable si lo justificas).
4. **Mensajes**: intercambia al menos 5 mensajes cifrados con AEAD, con contador/nonce seguro (p. ej. nonce derivado + contador).

Debes incluir defensa contra:

- Modificación de mensajes (debe detectarse).
- Replays simples (al menos con contador).

3) Requisitos no funcionales (obligatorios)

3.1. Buenas prácticas criptográficas

- Prohibido “RSA sin padding”. En RSA usa:
 - **OAEP** para cifrado,
 - **PSS** para firmas (si usas RSA para firmas).

- Usa AEAD (GCM o ChaCha20-Poly1305). No uses CBC+HMAC “a mano” salvo que lo justifiques mucho.
- No reutilices nonces en AEAD. Documenta cómo los generas.
- Borra o minimiza secretos en logs; añade un modo --verbose que nunca muestre claves privadas.

3.2. Pruebas

Incluye (mínimo):

- **10 pruebas unitarias** (keys + KDF + formatos + errores típicos).
- **6 pruebas de integración** (encrypt→decrypt, sign→verify, handshake→mensajes).
- Al menos **4 pruebas negativas** (ciphertext modificado, tag incorrecto, firma incorrecta, contraseña incorrecta).

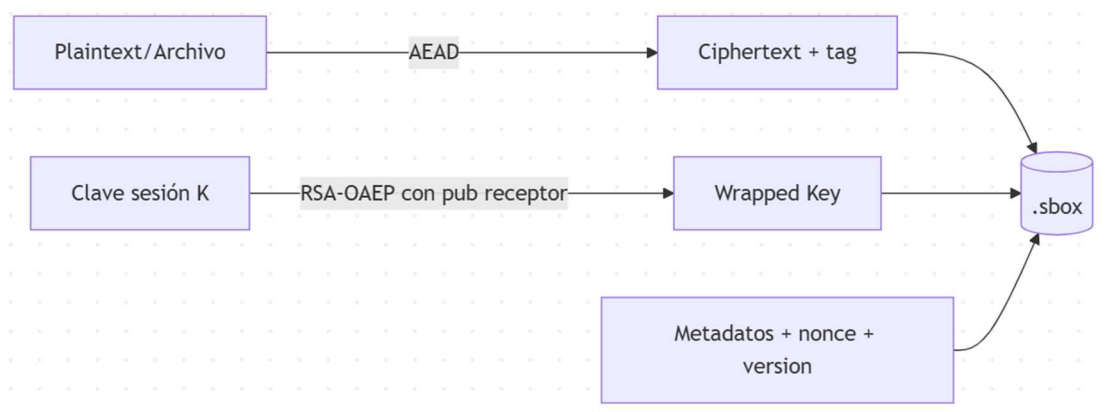
3.3. Usabilidad

- --help en todos los comandos.
- Mensajes de error claros (qué falló y por qué).
- securebox inspect archivo.sbox imprime un resumen legible sin revelar secretos.

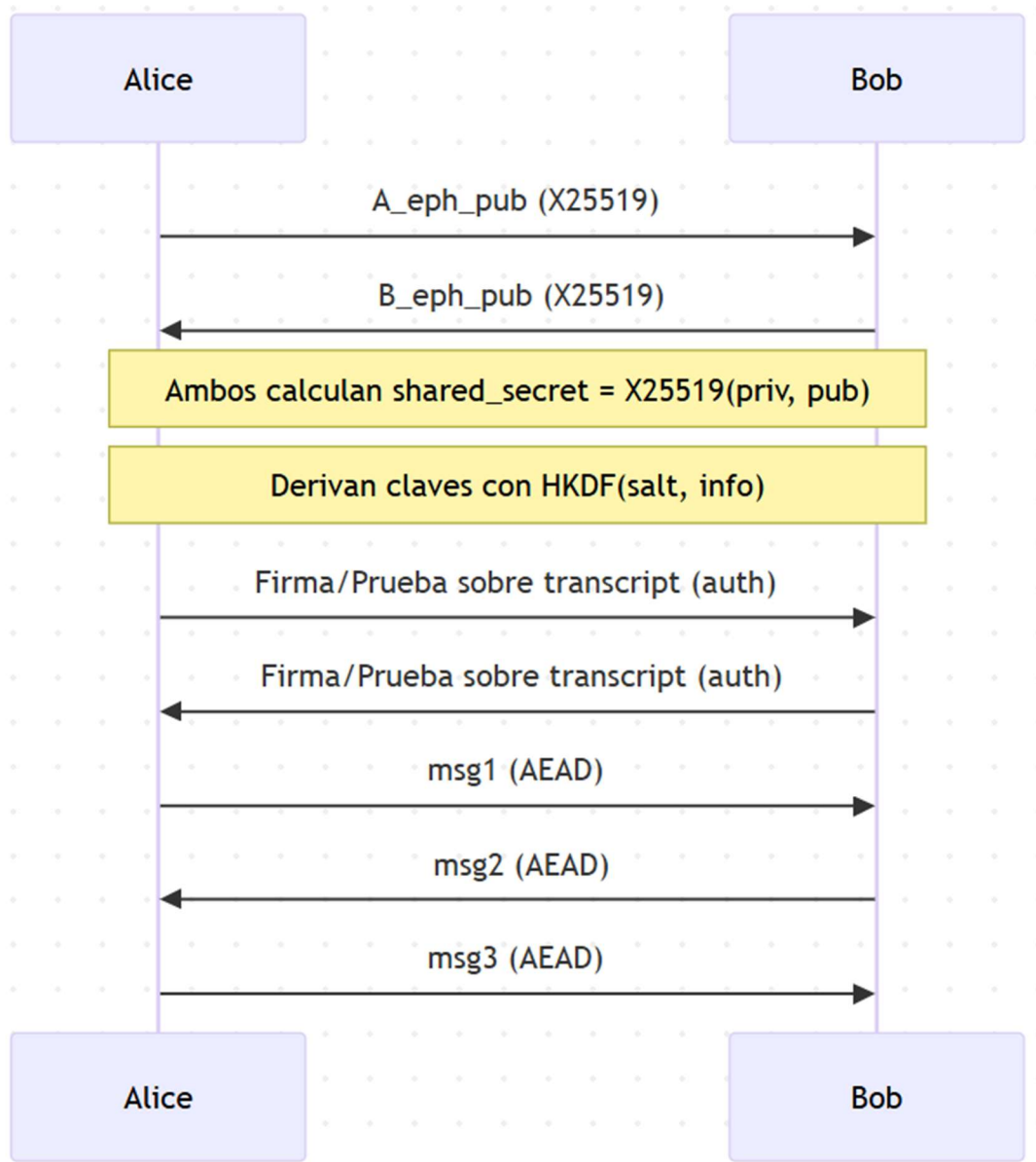
4) Diagramas (orientativos)

4.1. Cifrado híbrido (RSA Envelope)

f



4.2. Handshake (ECDHE + HKDF + AEAD)



5) Entregables

5.1. Código

- Proyecto completo con:
 - Código fuente
 - Tests ejecutables

- requirements.txt o pyproject.toml
- Instrucciones de ejecución (docs/USO.md)

5.2. Informe (8–12 páginas)

Incluye, como mínimo:

1. **Introducción** (qué implementas y por qué)
2. **Diseño del contenedor .sbox** (campos, decisiones, versión)
3. **Gestión de claves** (tipos, serialización, almacenamiento)
4. **Cifrado híbrido**: Modo A y Modo B (diagrama, algoritmo, padding, HKDF, nonces)
5. **Firmas** (qué firmas, cómo canonicalizas, cómo verificas)
6. **Handshake y canal seguro** (transcript, derivación, anti-replay)
7. **Pruebas** (tabla de casos y resultados; capturas relevantes)
8. **Rendimiento** (mediciones simples: tamaño de contenedor, tiempo cifrado/descifrado)
9. **Dificultades y lecciones aprendidas** (fallos típicos: padding, nonces, formatos)
10. **Conclusiones**

5.3. Ejecución reproducible

Incluye una sección “**Cómo ejecutar**” con comandos exactos, por ejemplo:

- Crear claves
- Cifrar un archivo en modo RSA
- Cifrar en modo ECC
- Firmar y verificar
- Ejecutar demo de handshake
- Correr tests

6) Rúbrica (sobre 100 puntos)

A) Implementación y corrección (50 pts)

- Gestión de claves completa y segura (10)
- Modo A (RSA envelope) correcto (15)
- Modo B (ECC KEM/DEM) correcto (15)
- CLI funcional y usable (10)

B) Seguridad aplicada (25 pts)

- Uso correcto de padding/AEAD/HKDF, nonces, validaciones (15)
- Firma/verificación robusta y ligada a manifest/transcript (10)

C) Pruebas y calidad (15 pts)

- Cobertura de tests (10)
- Manejo de errores y casos negativos (5)

D) Documentación (10 pts)

- Informe claro y completo (8)
- USO.md y ejemplos reproducibles (2)

7) Sugerencia sobre la secuencia a seguir

- Empieza por el **formato .sbox** y el comando inspect (te fuerza a definir campos).
- Luego implementa **Modo A** (RSA+AEAD) y tests de ida/vuelta.
- Después implementa **firmas** y tests negativos.
- Luego haz **Modo B** (ECDH+HKDF+AEAD).
- Por último, el **handshake demo**.